

Quorum

AI-Powered GitHub Pull Request Review and Analysis System

Software Project Report

Saiful Alam (0112320105)
Sabbir Ahmed (011222279)

CSE 3422 — Software Engineering Laboratory, Section C
Faculty: Zobaer Ibn Razzaque

October 3, 2026

Contents

1	Project Introduction / Objective	4
1.1	What Quorum Is	4
1.2	The Problem Addressed	4
1.3	What Quorum Automates	4
1.4	How the Result Helps Developers	4
1.5	The Role of the Dashboard	5
1.6	Project Objective	5
1.7	Implementation Status	5
2	Functional Requirements	5
2.1	A. Authentication and Authorization	5
2.2	B. GitHub Integration	6
2.3	C. Repository Management	6
2.4	D. Pull Request Processing	6
2.5	E. Automated Analysis Pipeline	6
2.6	F. Security Analysis	7
2.7	G. Test Generation and Execution	7
2.8	H. Analysis Results and Scoring	7
2.9	I. Dashboard	7
2.10	J. Ask Quorum	8
2.11	K. Review History	8
2.12	L. Settings and Profile	8
3	Scope of the Project	8
3.1	In Scope	8
3.2	Out of Scope	9
4	Technology Stack	10
4.1	LLM Configuration	10
5	Software Architecture	11
5.1	Overview	11
5.2	Component Description	11
5.3	Why the Backend Is the System Boundary	12
6	User Flow	12

6.1	High-Level User Flow	12
6.2	Step-by-Step Explanation	14
7	Use Case Definition	14
7.1	Actors	14
7.2	Use Case Diagram	14
7.3	Use Case Definitions	15
7.3.1	UC-01 — Authenticate with GitHub	15
7.3.2	UC-02 — Connect / Disconnect Repository	15
7.3.3	UC-03 — View Repositories / Workspace	16
7.3.4	UC-04 — View Pull Requests	16
7.3.5	UC-05 — Create Pull Request	16
7.3.6	UC-06 — Merge / Close Pull Request	16
7.3.7	UC-07 — View Analysis Results	17
7.3.8	UC-08 — View Review History	17
7.3.9	UC-09 — Ask Quorum	17
7.3.10	UC-10 — Settings / Sign Out	17
8	UI Screenshots	17

1 Project Introduction / Objective

1.1 What Quorum Is

Quorum is an AI-powered GitHub Pull Request review system. It is a web application that automatically analyzes pull requests opened in a GitHub repository, produces a deterministic Merge Readiness Score, and presents the results in a web dashboard with an evidence-grounded question-answering assistant called *Ask Quorum*. The system is implemented as a Python / FastAPI backend, a React/TypeScript frontend, and a PostgreSQL database, and it integrates with GitHub through a GitHub App, OAuth authentication, and webhooks.

1.2 The Problem Addressed

Reviewing a pull request manually is time-consuming and error-prone. A reviewer must inspect the changed code, look for security problems, estimate whether the change might break existing behaviour, and form an opinion about whether the change is safe to merge. This judgement is often based on subjective impressions rather than measured evidence, and it is easy to miss a security issue or a failing test. The conventional review process also requires the reviewer to either write and run tests manually or trust the author's claims.

1.3 What Quorum Automates

Quorum automates the evidence-gathering part of code review. When a pull request event reaches the backend, the orchestrator runs a review pipeline that:

- extracts and parses the pull request diff and the changed Python files (diff and AST analysis);
- runs the **Semgrep** static analysis tool over the changed files to obtain security evidence;
- passes the Semgrep findings and code context to a **Security Agent** (an LLM-driven agent) which reasons over the evidence and produces structured, evidence-traceable security findings;
- uses a **Test Writer Agent** to generate `pytest` tests for the modified functions;
- executes the generated tests inside an isolated, hardened **Docker sandbox** and measures code coverage before and after;
- combines the security, test, and coverage results into a deterministic **Merge Readiness Score** from 0 to 100;
- persists the results in PostgreSQL and posts a concise review summary back to the pull request on GitHub.

1.4 How the Result Helps Developers

The Merge Readiness Score is deterministic: the same evidence always produces the same score, and the breakdown (security findings, generated tests, and coverage) is stored with the analysis. A developer can therefore look at a pull request and immediately understand both the score and the reasons behind it, rather than relying on an unverifiable subjective opinion. The dashboard

shows security findings with evidence, generated test outcomes, coverage deltas, and the score for each pull request, and the Ask Quorum assistant answers questions about a specific review using only that review’s stored evidence.

1.5 The Role of the Dashboard

The React dashboard is the visual control centre for the pipeline. It allows a signed-in user to discover and connect GitHub repositories, open a repository workspace, create and manage pull requests (including creating, merging, and closing them from Quorum), view per-pull-request analysis results, review history, security findings, and settings, and ask questions about a review.

1.6 Project Objective

The overall objective of the project is to provide an end-to-end, evidence-based pull request review tool that helps developers assess whether a change is safe to merge. The project demonstrates the combination of static security analysis, LLM-driven reasoning, automated test generation, sandboxed test execution, coverage measurement, and deterministic result synthesis, all presented through a multi-user web application integrated with GitHub.

1.7 Implementation Status

All planned roadmap phases (Phase 0 through Phase 19) are implemented. The LLM layer is active through the OpenAI provider; a local `Ollama` provider exists in the codebase but is not currently used. Components that are planned but not yet implemented (such as an automated CI pipeline and a backup demo video) are noted in the project documentation as future work.

2 Functional Requirements

The functional requirements below describe the behaviour that is currently implemented in the Quorum codebase. Each requirement is identified by a code (**FR-NN**) and its implementation status is stated explicitly. Nothing is listed here that is not actually implemented.

2.1 A. Authentication and Authorization

ID	Requirement	Status
FR-01	The system shall allow a user to sign in using GitHub OAuth through the <i>Continue with GitHub</i> flow.	Implemented
FR-02	The system shall create a server-side session after a successful OAuth callback and clear it on logout.	Implemented
FR-03	The system shall expose the current user’s profile through <code>GET /api/me</code> .	Implemented
FR-04	The system shall accept the GitHub App installation callback (<code>GET /auth/install-callback</code>) to record the installation and return the user to Quorum with a connection result.	Implemented

2.2 B. GitHub Integration

ID	Requirement	Status
FR-05	The system shall verify every GitHub webhook request using the HMAC SHA-256 signature (<code>X-Hub-Signature-256</code>).	Implemented
FR-06	The system shall handle <code>pull_request</code> webhook events (<code>opened</code> , <code>reopened</code> , <code>synchronize</code> , <code>closed</code>), persist the pull request, and schedule analysis in the background.	Implemented
FR-07	The system shall use the GitHub App installation token for installation-scoped GitHub API calls.	Implemented
FR-08	The system shall post a concise review summary comment back to the pull request on GitHub after analysis.	Implemented
FR-09	The system shall create real pull requests on GitHub from the Quorum dashboard (<code>POST /api/repositories/{id}/pull-requests</code>).	Implemented
FR-10	The system shall merge and close pull requests on GitHub from the Quorum dashboard, with an optional merge message.	Implemented

2.3 C. Repository Management

ID	Requirement	Status
FR-11	The system shall list the user's GitHub repositories with their connected status (<code>GET /api/repositories/discover</code>).	Implemented
FR-12	The system shall connect and disconnect repositories through the GitHub App installation flow, using a popup and confirmation banners.	Implemented
FR-13	The system shall remember intentional disconnects so that automatic re-syncs do not re-attach a repository.	Implemented
FR-14	The system shall provide a per-repository workspace page that lists the repository's pull requests and offers a create-pull-request entry point.	Implemented

2.4 D. Pull Request Processing

ID	Requirement	Status
FR-15	The system shall list the branches of a connected repository (<code>GET /api/repositories/{id}/branches</code>).	Implemented
FR-16	The create-pull-request form shall let the user choose a Base and a Compare branch and shall auto-suggest a title from the compare branch.	Implemented
FR-17	The system shall reconcile a pull request's state from GitHub when the stored copy is stale, so that pull requests closed or merged on GitHub appear correctly in Quorum.	Implemented

2.5 E. Automated Analysis Pipeline

ID	Requirement	Status
FR-18	The system shall run the review pipeline through an orchestrator with registered stages and a persistent analysis-run lifecycle.	Implemented
FR-19	The system shall fetch and parse the unified diff of a pull request.	Implemented
FR-20	The system shall parse changed Python files with the standard <code>ast</code> module to identify modified functions and classes.	Implemented
FR-21	The system shall build a bounded, prioritized context from the changed files (context-window management).	Implemented

2.6 F. Security Analysis

ID	Requirement	Status
FR-22	The system shall run <code>Semgrep</code> over the changed files and parse its JSON findings into a structured evidence model.	Implemented
FR-23	The system shall run a Security Agent that reasons over the Semgrep evidence and the code context, and whose findings must be traceable to that evidence.	Implemented
FR-24	The system shall persist security findings with severity, file, line, evidence, and explanation.	Implemented

2.7 G. Test Generation and Execution

ID	Requirement	Status
FR-25	The system shall generate <code>pytest</code> tests for modified Python functions using a Test Writer agent, with syntax validation.	Implemented
FR-26	The system shall execute generated tests inside a hardened Docker sandbox (no network, memory/process limits, hard timeout, cleanup).	Implemented
FR-27	The system shall measure code coverage before and after the generated tests and compute the coverage delta.	Implemented

2.8 H. Analysis Results and Scoring

ID	Requirement	Status
FR-28	The system shall compute a deterministic Merge Readiness Score from 0 to 100 from the security, test, and coverage evidence, without LLM involvement.	Implemented
FR-29	The system shall persist the score and its breakdown (security, test, and coverage deductions) with the analysis run.	Implemented
FR-30	For pull requests with no changed Python files (for example documentation-only changes), the system shall skip the test and coverage deductions because they are not applicable.	Implemented

2.9 I. Dashboard

ID	Requirement	Status
FR-31	The dashboard shall show overview metrics, the user's repositories, and pull requests that need attention.	Implemented
FR-32	The dashboard shall list pull requests with their latest analysis status and score, and link to a detailed review page.	Implemented
FR-33	The dashboard shall display security findings across repositories and the review history.	Implemented
FR-34	The repositories page shall force a fresh sync with GitHub when opened, and background pages shall auto-refresh so that new results appear without a manual refresh.	Implemented

2.10 J. Ask Quorum

ID	Requirement	Status
FR-35	The system shall provide a chat endpoint scoped to a specific review (GET/POST /api/reviews/{id}/chat).	Implemented
FR-36	The assistant shall answer only from the selected review's stored evidence (findings, tests, coverage, and score breakdown) and treat repository content as untrusted input (prompt-injection defense).	Implemented
FR-37	The system shall persist chat messages and provide a full <i>Ask Quorum</i> page as well as a floating chat.	Implemented

2.11 K. Review History

ID	Requirement	Status
FR-38	The system shall list all analysis runs (reviews) with their score, status, findings, tests, and coverage, and provide a review detail page.	Implemented

2.12 L. Settings and Profile

ID	Requirement	Status
FR-39	The system shall show the user's GitHub account, connected repositories, and GitHub App authorization status, and allow the user to sign out.	Implemented

3 Scope of the Project

3.1 In Scope

The current Quorum project delivers the following, all of which are implemented in the repository:

- GitHub OAuth authentication and session management for the web application.

- GitHub App installation/authorization for repository access, including the installation callback, repository discovery, and one-click connect/disconnect flows with confirmation banners.
- Repository workspaces and pull request management from the dashboard, including creating pull requests (Base and Compare branches with suggested titles), and merging and closing pull requests with an optional merge message.
- Automated pull request analysis: diff extraction, Python AST analysis, bounded context management, Semgrep static security scanning, an LLM-driven Security Agent, an LLM-driven Test Writer, and sandboxed test execution with coverage measurement.
- A deterministic Merge Readiness Score (0–100) with a persisted breakdown, and an automatic GitHub review comment.
- PostgreSQL persistence of users, repositories, pull requests, analysis runs, security findings, test runs, coverage results, and chat messages, managed through Alembic migrations.
- A FastAPI REST API used by the dashboard for all reads and writes, with user-level access isolation.
- A React/TypeScript dashboard with landing page, dashboard, repositories, repository workspace, pull requests, PR review details, findings, review history, profile, settings, and Ask Quorum.
- An evidence-grounded Ask Quorum assistant scoped to the selected review.
- An evaluation harness (`evaluation/`) with deterministic metrics over a labeled pull request dataset.
- A local development setup using a Cloudflare quick tunnel to receive GitHub webhooks during development.

3.2 Out of Scope

The following are explicitly outside the scope of the current project and are not implemented:

- Replacing GitHub itself, or providing a Git hosting platform.
- Replacing human code review; Quorum provides evidence and a score but does not make a final merge decision for the user.
- Unrestricted repository administration, such as managing collaborators or repository settings.
- Full IDE functionality or arbitrary in-app code editing of the repository.
- A general-purpose chatbot; Ask Quorum is intentionally restricted to the evidence of the selected review.
- Automated CI/CD pipeline configuration (a basic CI pipeline is planned future work, not yet implemented).
- Automated deployment to a production hosting platform.
- Support for languages other than Python for test generation and coverage (Semgrep scanning is language-agnostic, but the AST, test writer, and coverage stages target Python).

These items are not claimed as implemented anywhere in this report; they are documented here so that the scope of the project is unambiguous.

4 Technology Stack

The table below lists the technologies actually used in the Quorum implementation. Each technology was verified against the repository (requirements files, configuration, and source code).

Layer	Technology	Usage
Backend language	Python 3.13	Main backend, analysis, orchestration
Backend framework	FastAPI	REST API and GitHub webhook server
ASGI server	Uvicorn	Runs the FastAPI application
Data validation	Pydantic	Request/response and model validation
Database	PostgreSQL	Persistent application/review/chat data
ORM	SQLAlchemy 2.0	Database access
Migrations	Alembic	Database schema migrations
GitHub integration	GitHub App, OAuth, Webhooks, REST API	PR events, auth, installation access
HTTP client	httpx	GitHub and LLM communication
JWT	PyJWT	GitHub App authentication
Static analysis	Semgrep (v1.177)	Security evidence over changed files
Code analysis	Python standard <code>ast</code>	Structural analysis of changed Python code
LLM interface	<code>LLMProvider</code> abstraction	Security, test, and chat reasoning
LLM provider (active)	OpenAI (GPT-5.6 Terra / Luna)	Security Agent, Test Writer, Ask Quorum
LLM provider (dormant)	Ollama (Qwen2.5-Coder)	Optional local fallback, not in use
Test execution	Docker sandbox	Isolated execution of generated tests
Sandbox image	<code>quorum-sandbox</code>	pytest + pytest-cov image
Backend tests	pytest, pytest-cov	Automated backend tests
Frontend	React, TypeScript	Web dashboard
Frontend build	Vite	Fast development/build
Styling	Tailwind CSS	UI styling
UI components	shadcn-style design tokens	Reusable accessible UI components
Frontend tests	Vitest, React Testing Library	Component tests
Development web-hook tunnel	Cloudflare Quick Tunnel	Expose local FastAPI to GitHub

4.1 LLM Configuration

The LLM is accessed through a common `LLMProvider` interface so that no agent depends on a concrete provider. The active provider is OpenAI, with per-role models:

- Security Agent → `gpt-5.6-terra`
- Test Writer → `gpt-5.6-terra`
- Ask Quorum → `gpt-5.6-luna`

A single `OPENAI_API_KEY` authenticates all roles and is stored only server-side. The `OllamaProvider` exists in the codebase as an optional local fallback but is not currently used.

5 Software Architecture

5.1 Overview

Quorum follows a strict layered architecture. The React dashboard is a presentation layer that communicates **only** with the FastAPI REST API; it never accesses PostgreSQL, Docker, Semgrep, the LLM, or GitHub server secrets directly. The backend is the system boundary: it owns authentication, authorization, GitHub integration, database access, analysis orchestration, LLM calls, and sandboxed test execution.

The main boundaries are:

- **External systems:** GitHub (OAuth, webhooks, REST API) and the Docker daemon that hosts the sandbox.
- **Application components:** the FastAPI backend (auth, webhook handler, REST API), the orchestrator review pipeline, and the analysis services (Semgrep, LLM-based Security and Test Writer agents).
- **Persistence:** PostgreSQL, accessed only by the backend.
- **Frontend:** the React dashboard, connected to the backend REST API only.

Figure 1 shows the layered architecture of the implemented system.

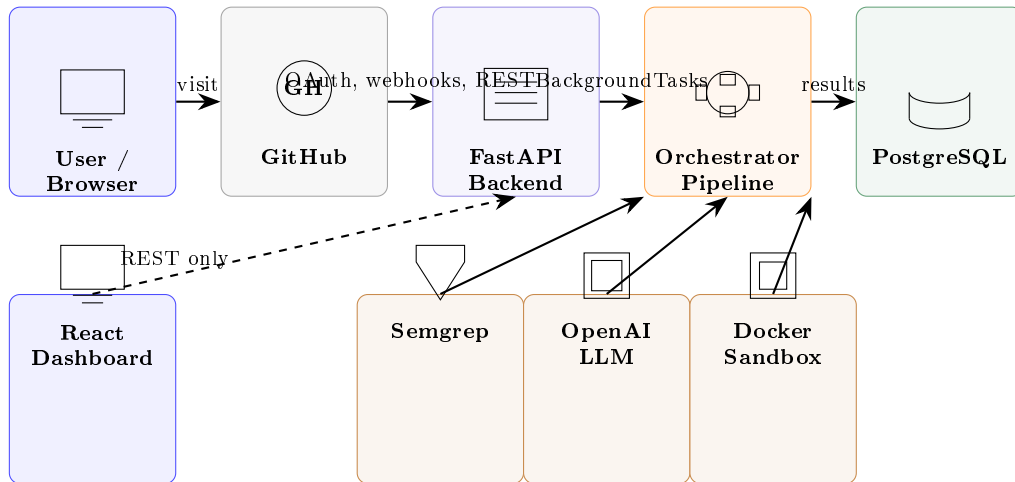


Figure 1: Quorum architecture as an icon-block flow. The dashboard talks only to the FastAPI backend; analysis services are driven by the orchestrator.

5.2 Component Description

GitHub is the source of truth for pull requests. It delivers `pull_request` webhook events, authorizes the user through OAuth, and provides installation-scoped repository access through the GitHub App.

FastAPI Backend exposes the OAuth flow (`/auth/*`), the webhook endpoint (`POST /webhooks/github`, HMAC-verified), and the dashboard REST API (`/api/*`). It enforces user-level access isolation for every read and write.

Orchestrator / Review Pipeline runs the registered analysis stages in a background task for each pull request. It creates an analysis run, marks it in progress, executes the stages, and records completion or failure.

Diff + AST fetch and parse the pull request diff and the changed Python files to identify modified functions and classes.

Context builds a bounded, prioritized prompt context from the changed files so that the LLM never receives an entire repository.

Semgrep scans the changed files and produces structured JSON findings that form the security evidence.

Security Agent (LLM) reasons over the Semgrep evidence and the code context, and produces structured findings that must be traceable to the evidence; it cannot invent findings.

Test Writer Agent (LLM) generates `pytest` tests for the modified functions.

Docker Sandbox executes the generated tests in a hardened container (no network, memory/process limits, hard timeout, guaranteed cleanup) and measures coverage before and after.

Synthesis combines the security, test, and coverage results into the deterministic Merge Readiness Score, persists it with its breakdown, and builds the review comment that is posted back to GitHub.

PostgreSQL persists all application data; it is accessed only by the backend.

React Web Dashboard displays data through the FastAPI REST API and never accesses the database, Docker, Semgrep, the LLM, or GitHub directly.

5.3 Why the Backend Is the System Boundary

All privileged operations (GitHub tokens, database access, sandbox execution, and LLM calls) require secrets or host access that must never be exposed to the browser. By enforcing that the dashboard talks only to the FastAPI REST API, the system keeps GitHub App tokens, the database, and the analysis infrastructure out of the frontend and behind a single authenticated boundary.

6 User Flow

6.1 High-Level User Flow

Figure 2 shows the user flow as a swimlane diagram with three lanes: the **User / Browser**, the **Quorum System**, and **GitHub**. GitHub remains the source of truth: the user leaves Quorum for GitHub OAuth and for the GitHub App installation flow, and GitHub delivers pull request events back to Quorum through webhooks.

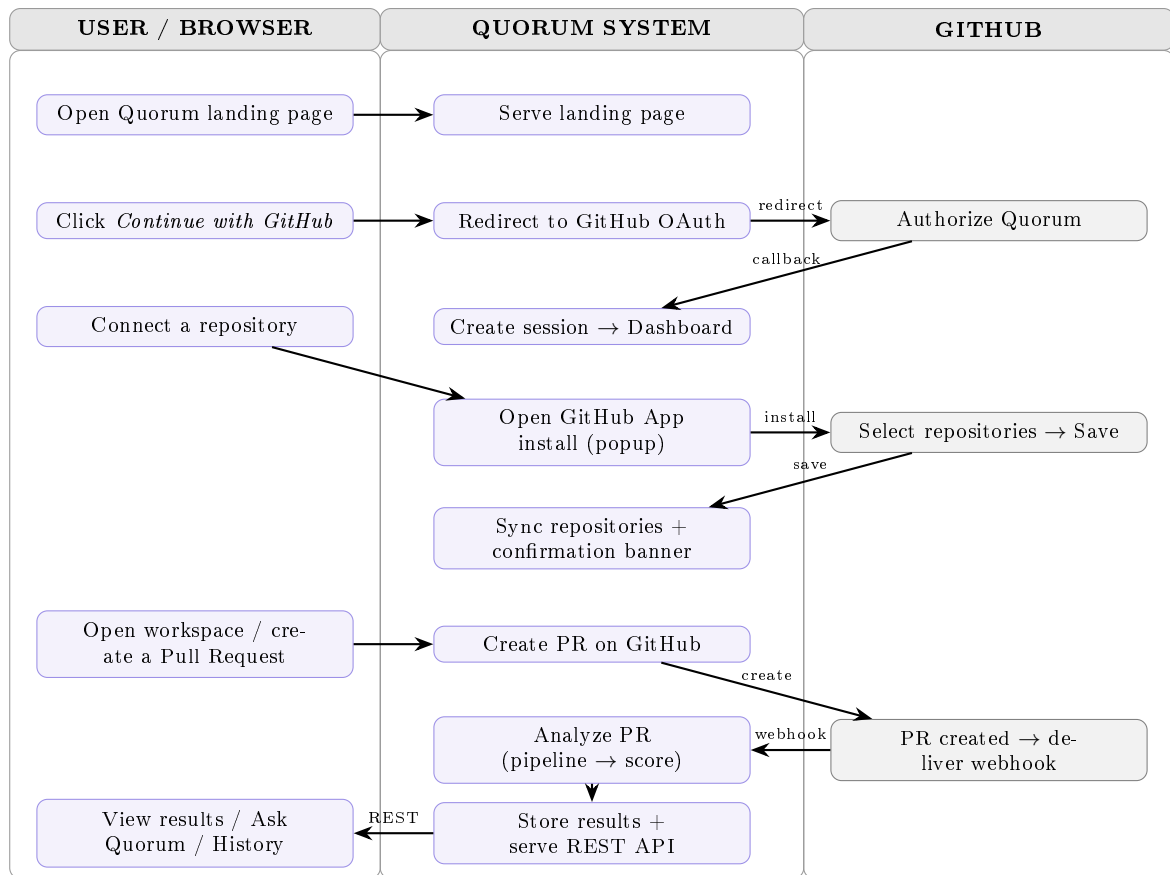


Figure 2: Swimlane user flow: the user alternates between Quorum and GitHub, and GitHub returns control and events to Quorum.

6.2 Step-by-Step Explanation

1. **Landing page.** A public landing page introduces the product before authentication.
2. **GitHub OAuth.** The user clicks *Continue with GitHub*, Quorum redirects to GitHub, and the user authorizes Quorum. GitHub redirects back and Quorum creates a server-side session.
3. **Quorum dashboard.** The signed-in user lands on the dashboard.
4. **Connect repository.** The user connects a repository. The GitHub App install page opens in a popup; when the user selects repositories and saves, GitHub redirects back, Quorum syncs the authorized repositories, and a confirmation banner appears.
5. **Repository workspace / create PR.** The user opens a repository and creates a pull request from Quorum (Base/Compare branches, suggested title) or opens an existing one.
6. **PR creation on GitHub.** Quorum creates the pull request on GitHub through the GitHub App API.
7. **Webhook delivery.** GitHub delivers a `pull_request` webhook to Quorum.
8. **Analysis.** The orchestrator runs the review pipeline and stores the results (score, findings, tests, coverage).
9. **Results.** The user views results, asks Ask Quorum questions, and returns to review history later.

7 Use Case Definition

7.1 Actors

The main actors of the system are:

Developer / User — the primary actor. A signed-in GitHub user who connects repositories, opens pull requests, and reviews analysis results.

GitHub — an external system that authenticates the user, hosts repositories and pull requests, and delivers webhook events.

GitHub App — the Quorum GitHub App installation that grants repository access; it participates in repository connection.

Internal components (the orchestrator, the analysis agents, the sandbox, and PostgreSQL) are not treated as actors; they are implementation services that run inside the Quorum system.

7.2 Use Case Diagram

Figure 3 shows the use case model. All use cases are supported by the current implementation.

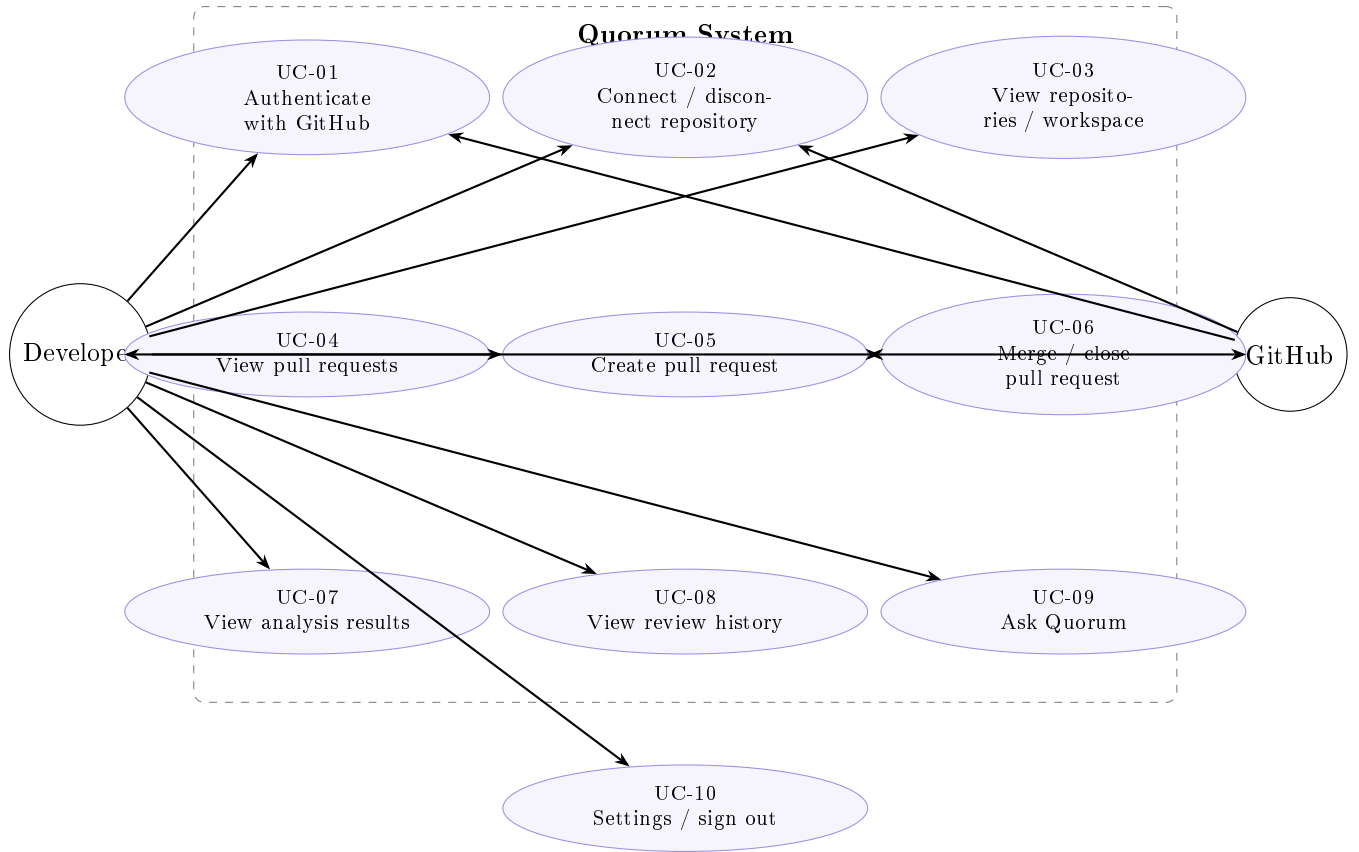


Figure 3: Quorum use case diagram.

7.3 Use Case Definitions

7.3.1 UC-01 — Authenticate with GitHub

Primary actor Developer.

Supporting system GitHub (OAuth).

Goal The developer signs in and Quorum creates a session.

Preconditions The developer has a GitHub account.

Trigger The developer clicks *Continue with GitHub*.

Main flow The developer authorizes Quorum on GitHub; Quorum receives the OAuth callback, persists the user, and creates a session.

Postconditions The dashboard shows the signed-in user.

7.3.2 UC-02 — Connect / Disconnect Repository

Primary actor Developer.

Supporting system GitHub App.

Goal Grant (or revoke) Quorum access to a repository.

Preconditions The developer is signed in.

Trigger The developer clicks Connect or Disconnect on the repositories page or workspace.

Main flow The GitHub App install/manage page opens in a popup; the developer selects repositories and saves; GitHub redirects back and Quorum syncs and shows a confirmation banner.

Postconditions The repository is connected (or disconnected and remembered as such).

7.3.3 UC-03 — View Repositories / Workspace

Primary actor Developer.

Goal Browse connected repositories and open a repository workspace.

Main flow The developer opens the Repositories page, sees connected and available repositories, and opens a repository to see its pull requests.

7.3.4 UC-04 — View Pull Requests

Primary actor Developer.

Goal See the pull requests of a repository with their analysis status.

Main flow The developer opens a repository or the Pull Requests page and reviews the list.

7.3.5 UC-05 — Create Pull Request

Primary actor Developer.

Supporting system GitHub.

Goal Create a real pull request on GitHub from Quorum.

Main flow The developer selects Base and Compare branches (a title is suggested), adds a description, and submits; Quorum creates the pull request on GitHub and starts analysis.

Alternative GitHub rejects the pull request; Quorum shows the error.

7.3.6 UC-06 — Merge / Close Pull Request

Primary actor Developer.

Supporting system GitHub.

Goal Merge or close a pull request from Quorum.

Main flow The developer chooses Merge (optionally with a message) or Close; Quorum performs the action on GitHub and the state is updated.

Alternative The merge fails (for example conflict); Quorum shows the GitHub error.

7.3.7 UC-07 — View Analysis Results

Primary actor Developer.

Goal See the Merge Readiness Score, security findings, generated tests, and coverage for a pull request.

Main flow The developer opens the PR review page; the results refresh automatically as analysis progresses.

7.3.8 UC-08 — View Review History

Primary actor Developer.

Goal Review previous analyses.

Main flow The developer opens Review History and a previous review detail.

7.3.9 UC-09 — Ask Quorum

Primary actor Developer.

Goal Ask questions about a specific review.

Main flow The developer asks a question on the review; the assistant answers using only that review's evidence.

7.3.10 UC-10 — Settings / Sign Out

Primary actor Developer.

Goal View account and connection status and sign out.

Main flow The developer opens Settings, reviews GitHub account and connected repositories, and signs out.

8 UI Screenshots

This section references screenshots of the actual Quorum application. The images are not included in the repository; they are placed manually into **report/screenshots/** using the filenames listed in *SCREENSHOTS_NEEDED.md*. If a file is missing, a placeholder box is shown instead so the report still compiles.

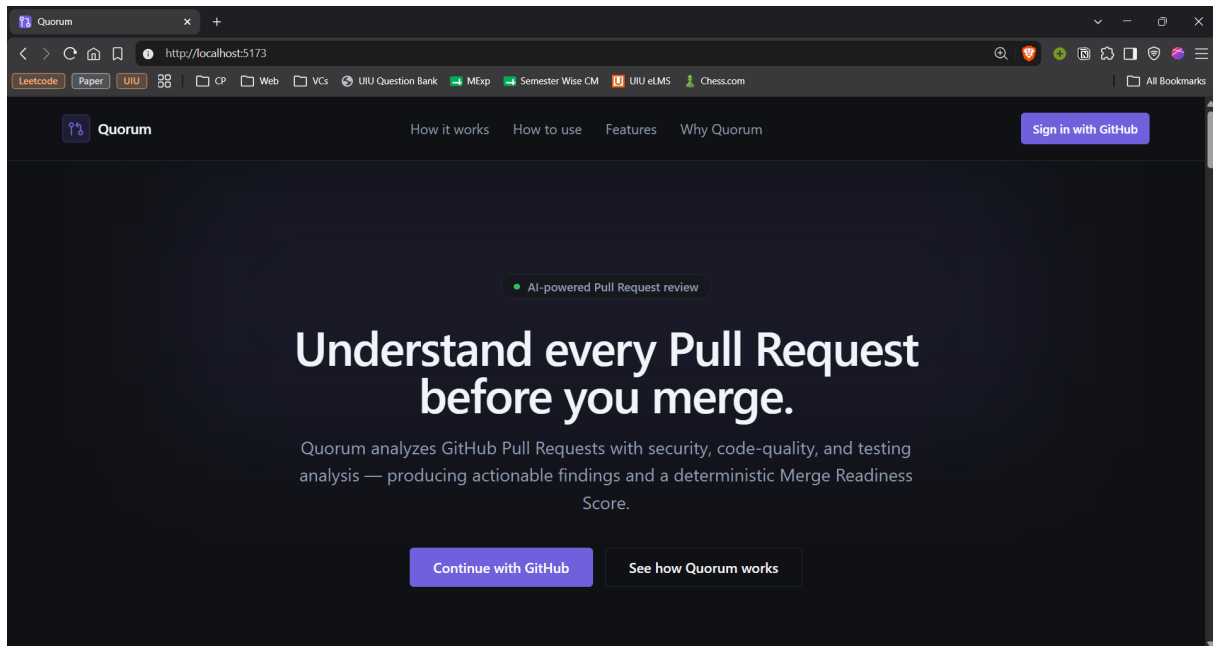


Figure 4: figure:screenshot-landing — The public landing page that introduces Quorum before authentication

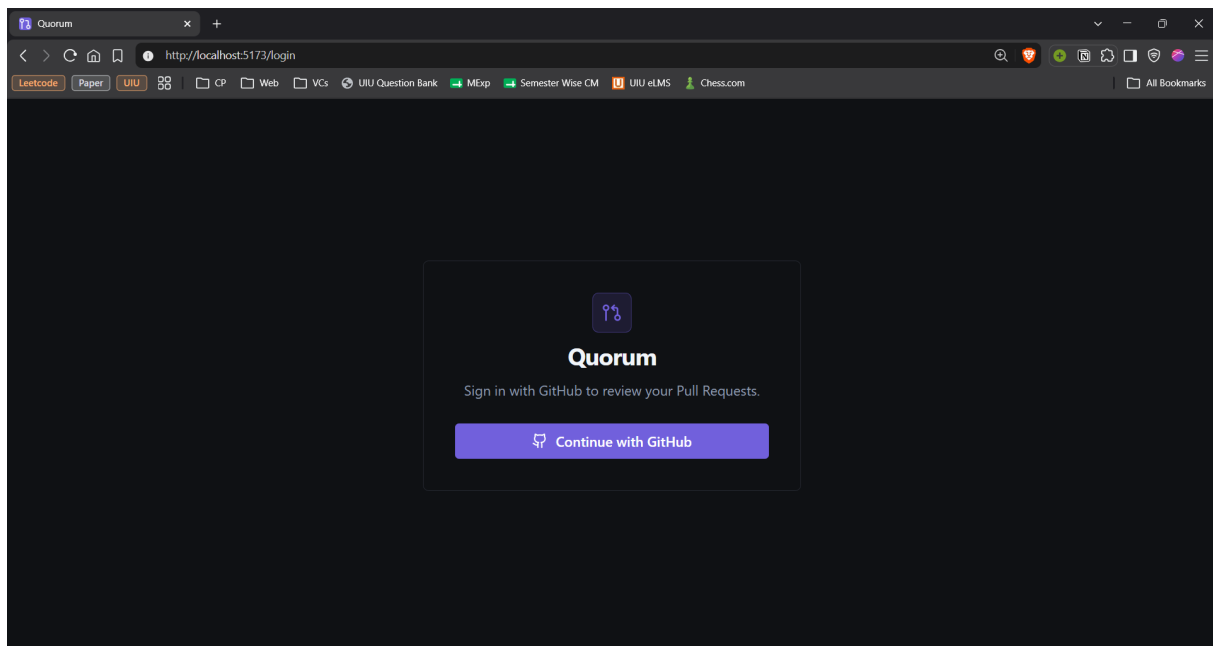


Figure 5: figure:screenshot-login — The sign-in transition to GitHub OAuth

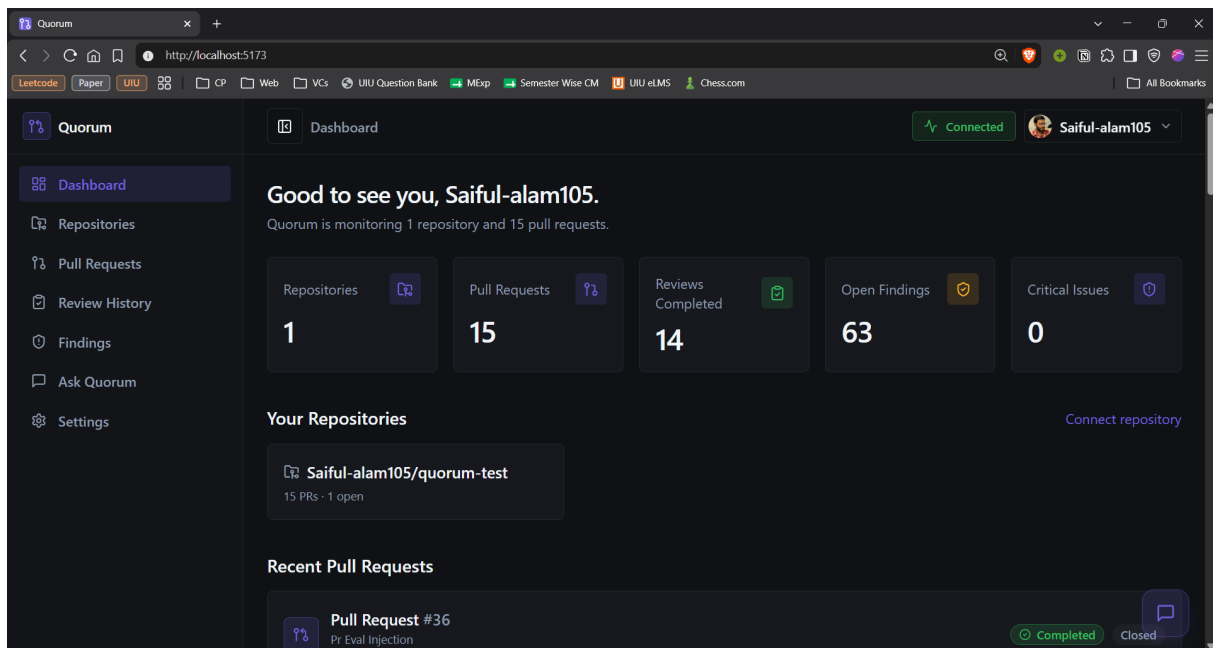


Figure 6: figure:screenshot-dashboard — The main dashboard with metrics, repositories, and pull requests

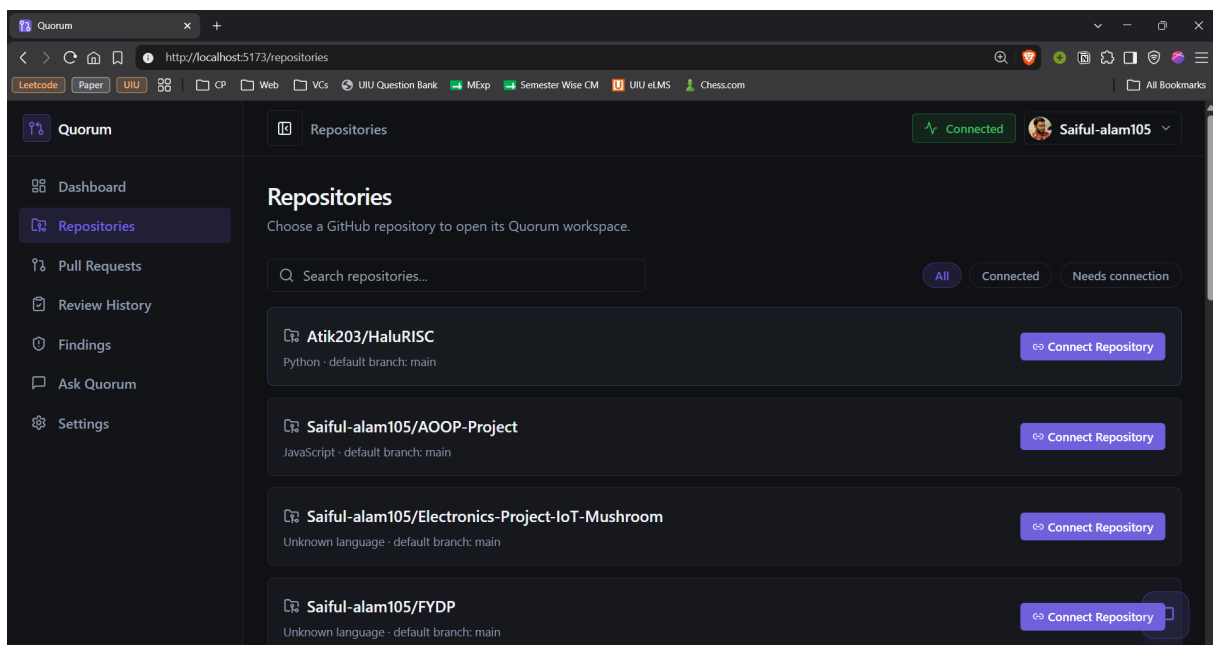


Figure 7: figure:screenshot-repositories — The repositories page with discovery and connection status

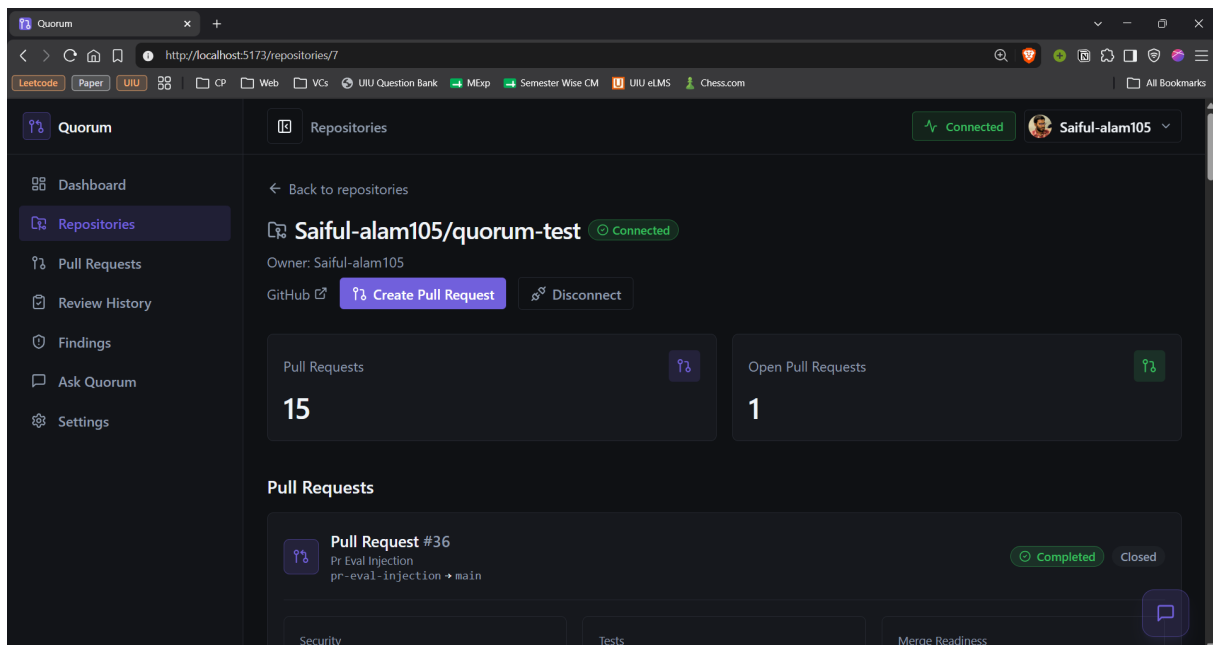


Figure 8: figure:screenshot-workspace — A connected repository's workspace showing its pull requests

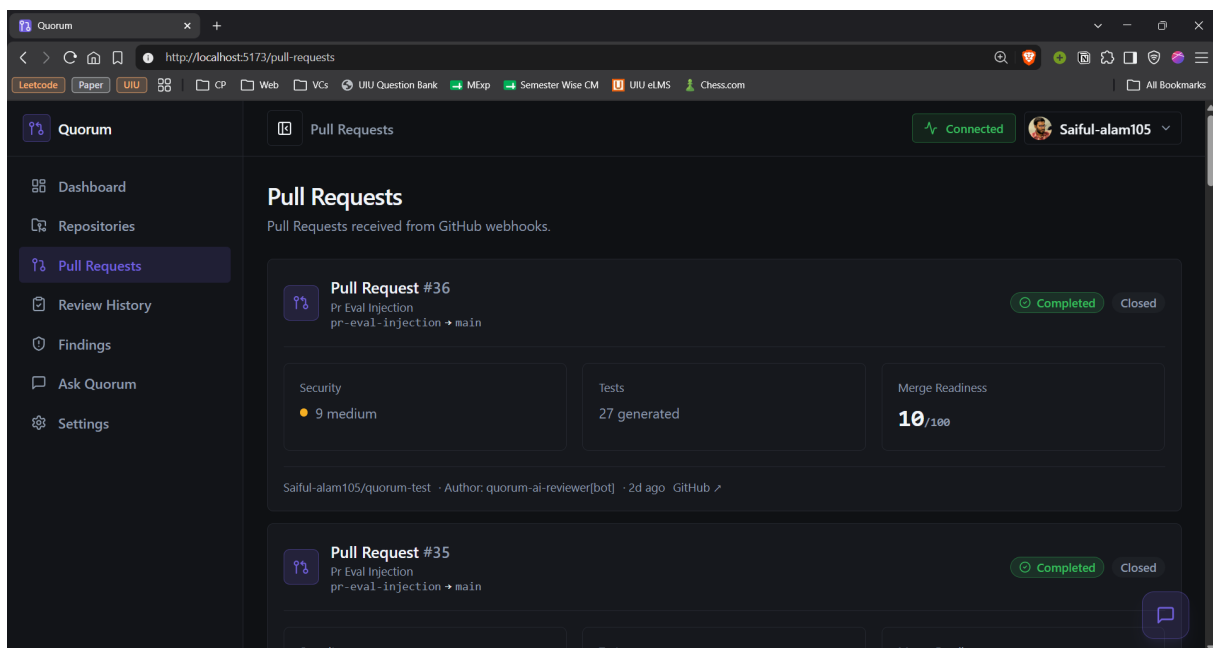


Figure 9: figure:screenshot-pull-requests — The pull request list with analysis status

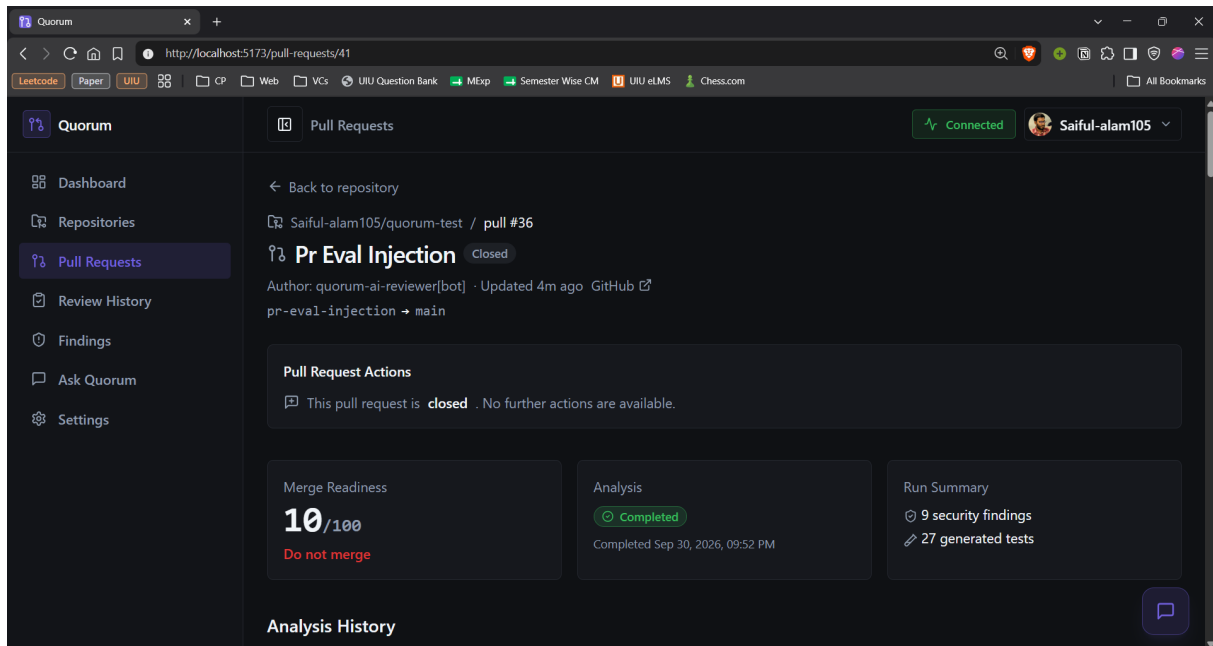


Figure 10: figure:screenshot-pr-review — The review page with score, findings, tests, and coverage

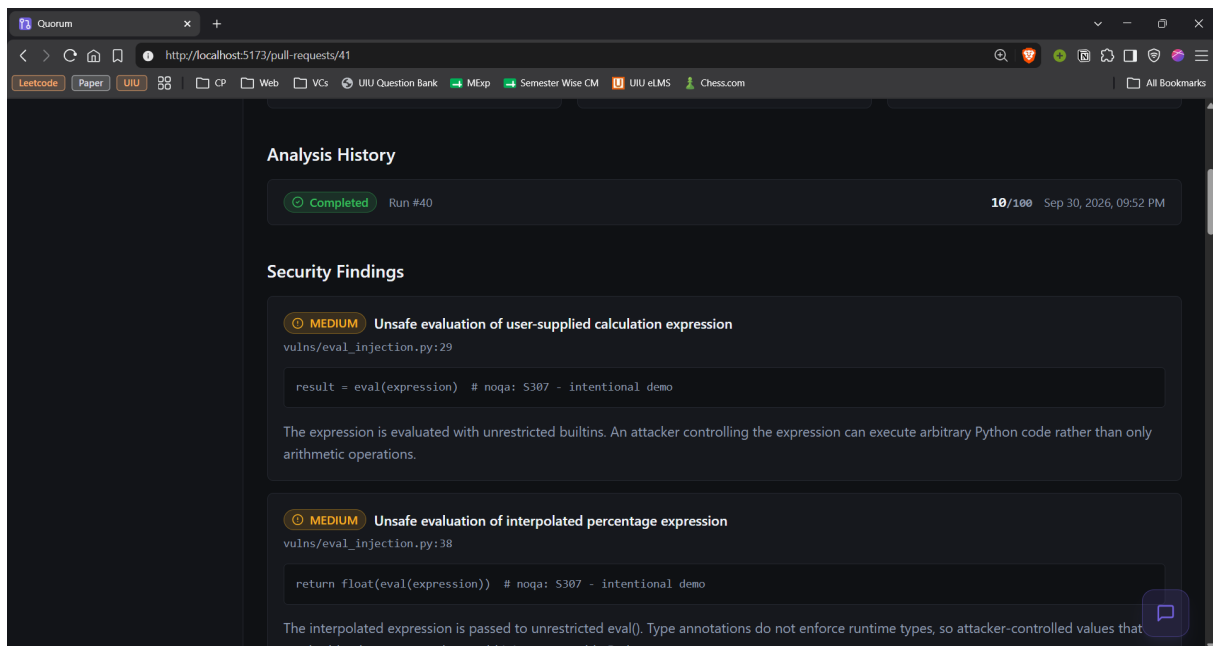


Figure 11: figure:screenshot-findings — Security findings with severity, location, and evidence

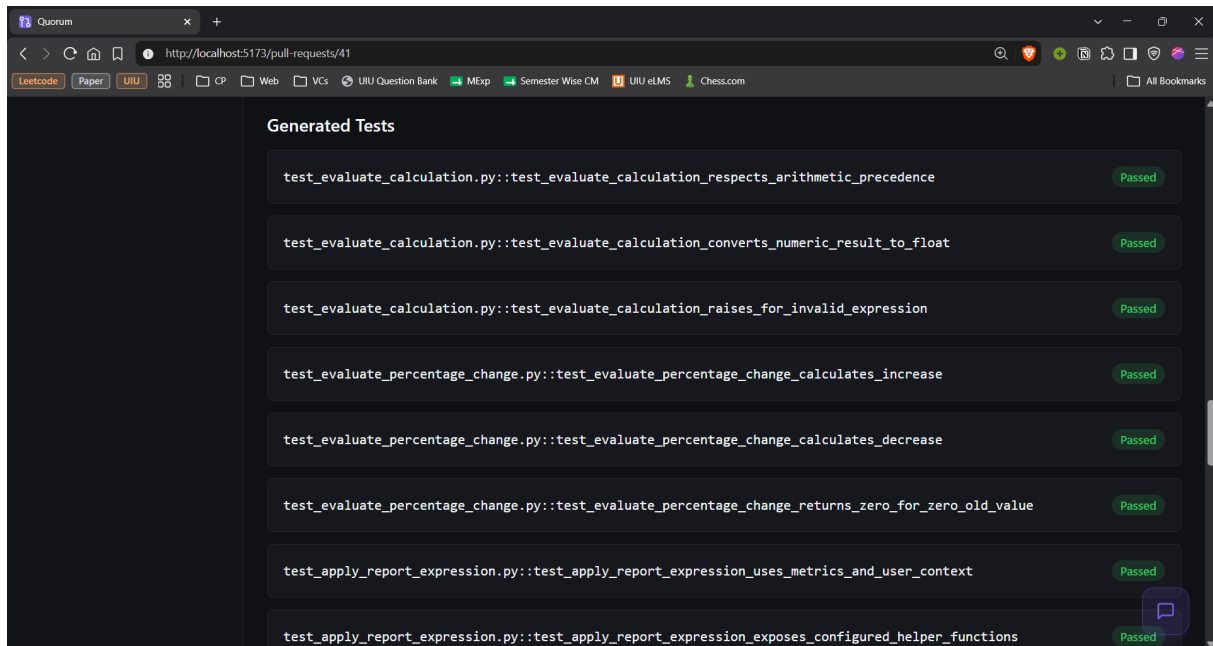


Figure 12: figure:screenshot-tests — Generated tests and their pass/fail outcomes

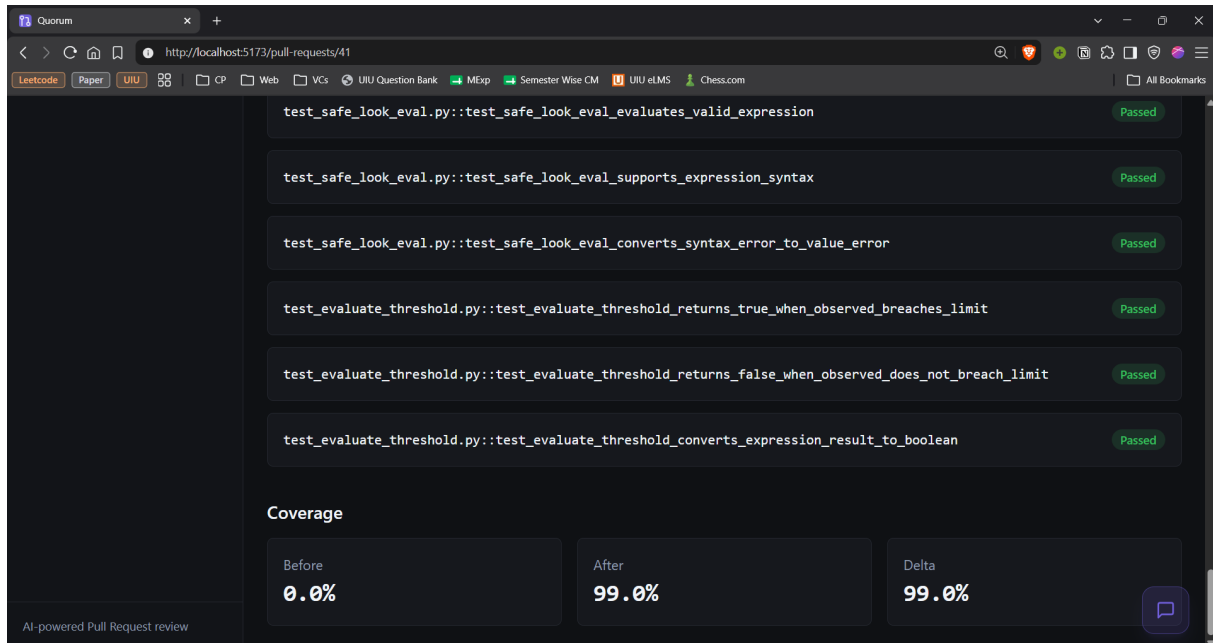


Figure 13: figure:screenshot-coverage — Coverage before/after and delta for a review

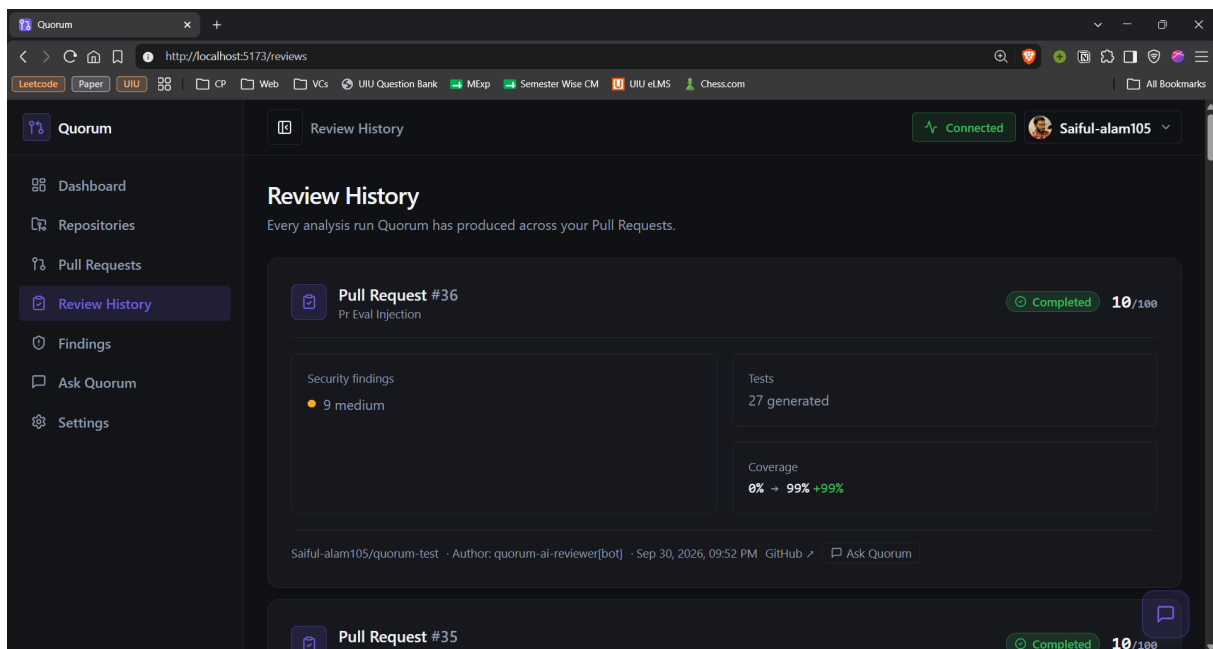


Figure 14: figure:screenshot-history — The list of previous analyses

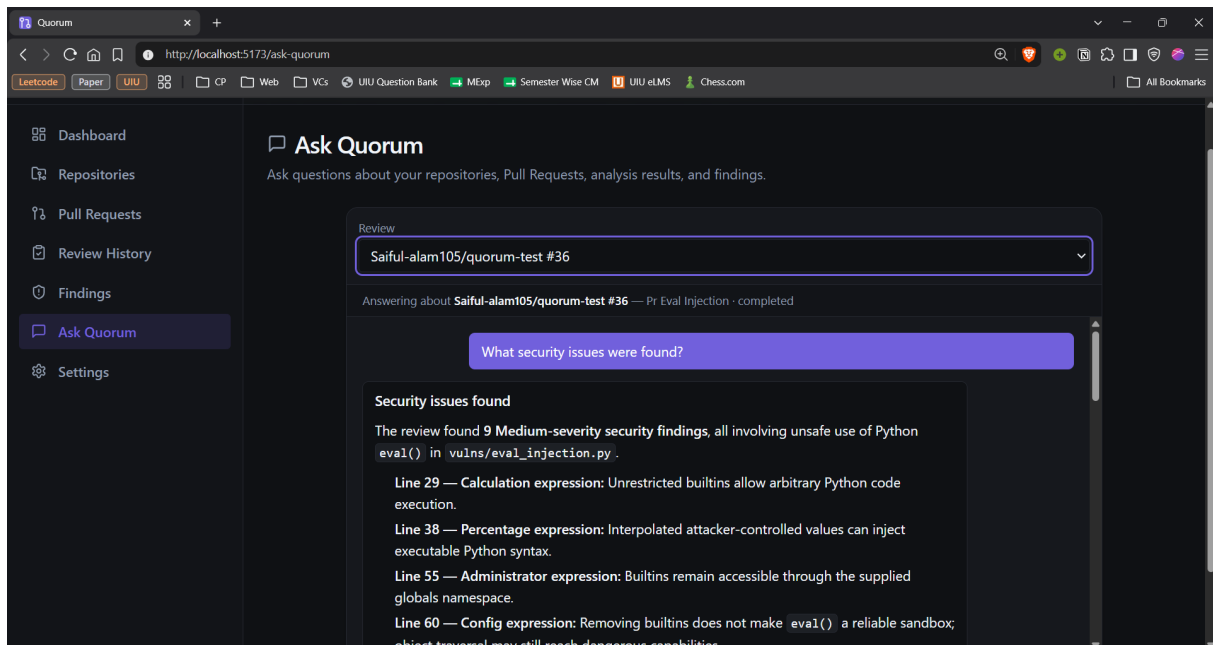


Figure 15: figure:screenshot-ask — The evidence-grounded question-answering assistant

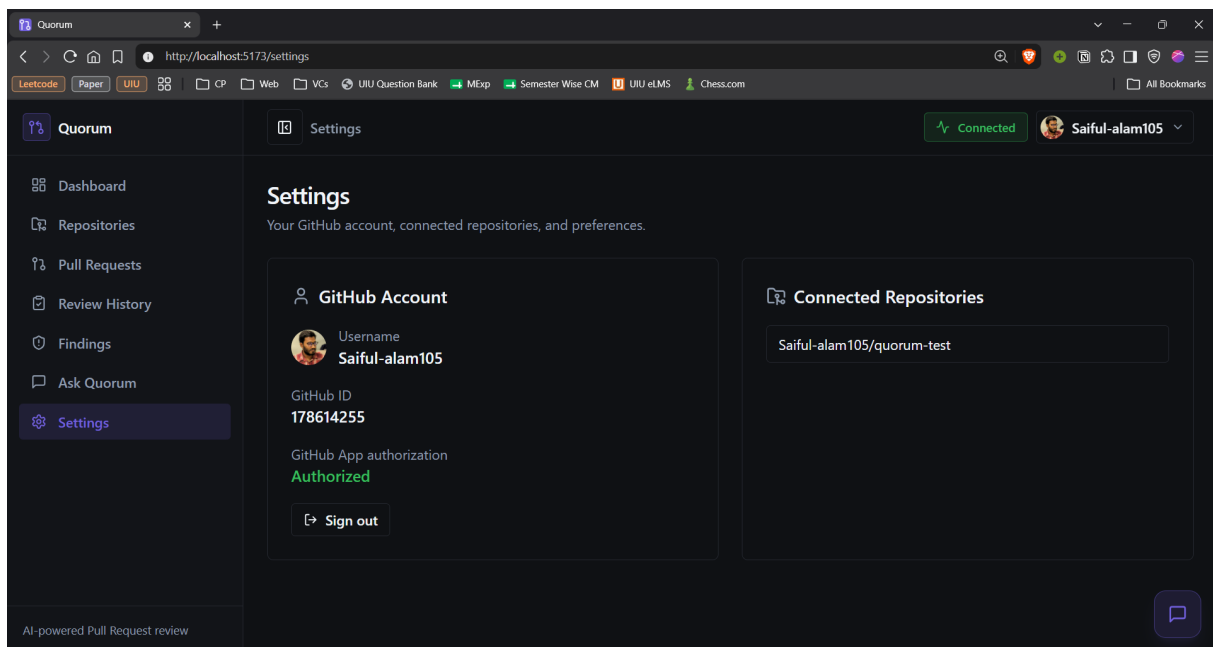


Figure 16: figure:screenshot-settings — GitHub account, connected repositories, and sign out